HEIG-VD / INGÉNIERIE DES MÉDIAS / PROGRAMMATION WEB

INTRODUCTION AU JAVASCRIPT ASYNCHRONE

HEIG VD

```
const paragraph = document.querySelector("p");

paragraph.textContent = "Sympa!";

alert("texte modifié");

paragraph.style.color = "red";
```



```
const paragraph = document.querySelector("p");

paragraph.textContent = "Sympa!";

alert("texte modifié");

paragraph.style.color = "red";
```



```
const paragraph = document.querySelector("p");

paragraph.textContent = "Sympa!";

alert("texte modifié");

paragraph.style.color = "red";
```



```
const paragraph = document.querySelector("p");

paragraph.textContent = "Sympa!";

alert("texte modifié");

paragraph.style.color = "red";
```



```
const paragraph = document.querySelector("p");

paragraph.textContent = "Sympa!";

alert("texte modifié");

paragraph.style.color = "red";
```





Le majeure partie du code écrit jusqu'à présent est synchrone:

- Il est exécuté ligne par ligne
- Le moteur attend que la ligne précédente aie fini d'être exécutée avant de passer à la ligne suivante.

😢 **Les longues opérations bloquent l'exécution du code!**

HEIG VD

```
const paragraph = document.querySelector(“p");


setTimeout(() ⇒ {
  paragraph.textContent = "Sympa!";
}, 5000);

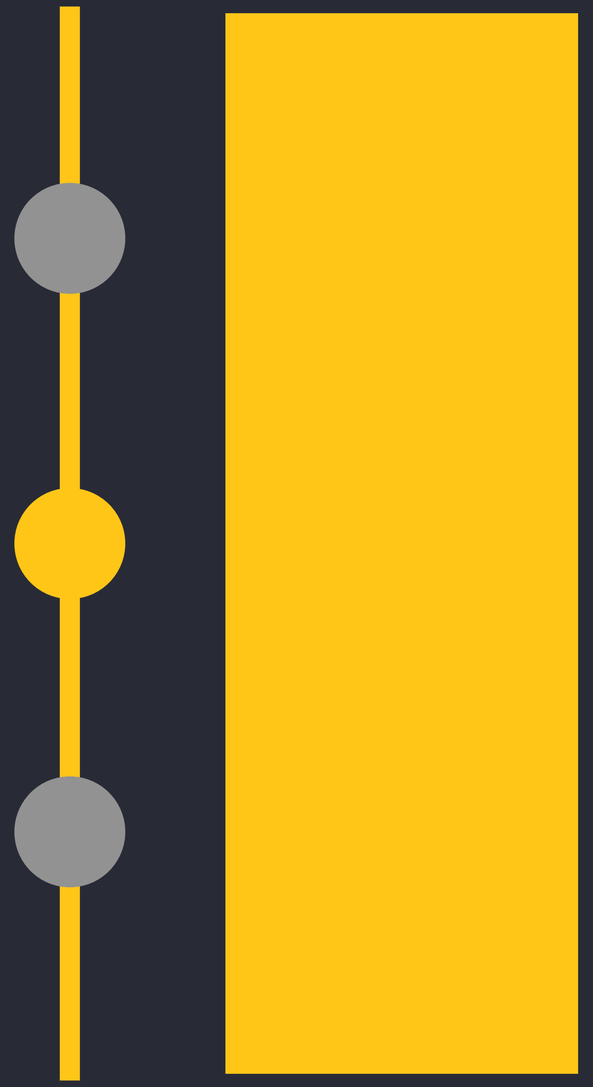

paragraph.style.color = "red";
```



```
const paragraph = document.querySelector(“p");


setTimeout(() ⇒ {
  paragraph.textContent = "Sympa!";
}, 5000);

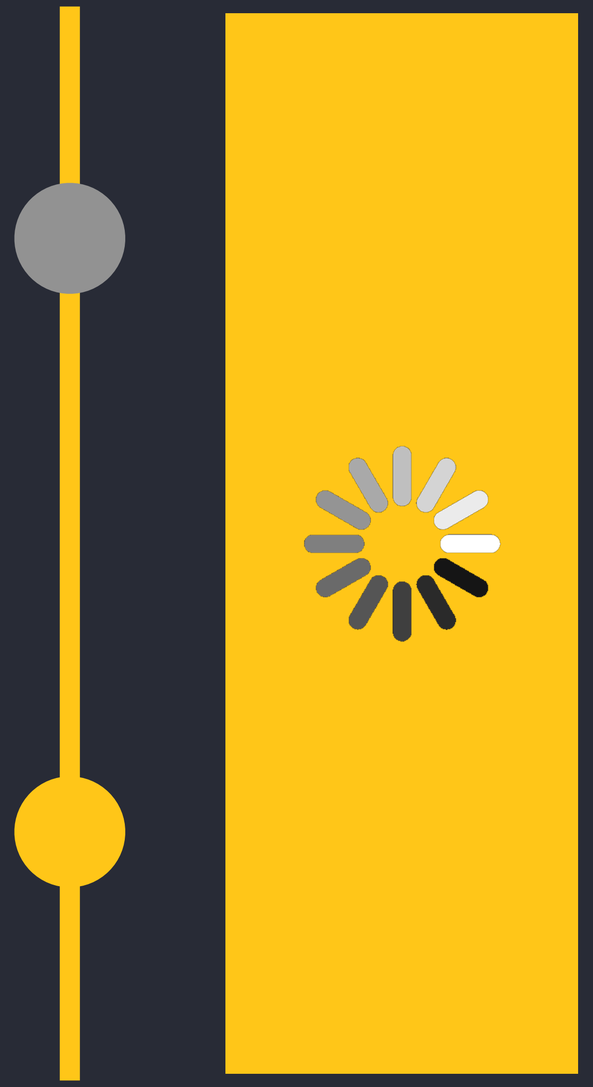

paragraph.style.color = "red";
```



```
const paragraph = document.querySelector(“p");


setTimeout(() ⇒ {
  paragraph.textContent = "Sympa!";
}, 5000);

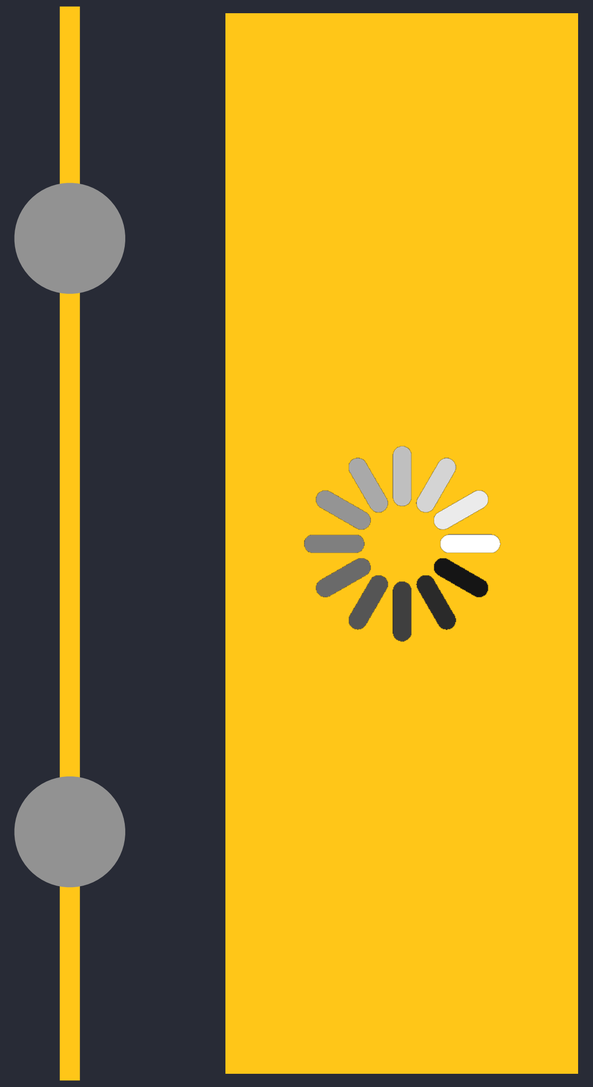

paragraph.style.color = “red";
```

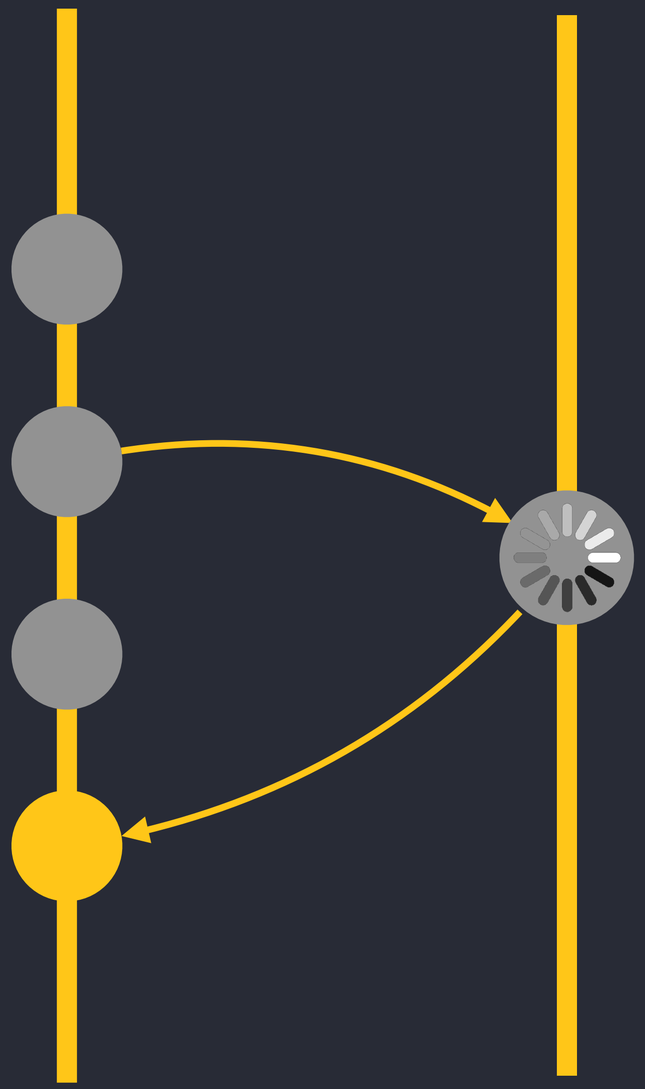


```
const paragraph = document.querySelector(“p");


setTimeout(() ⇒ {
  paragraph.textContent = "Sympa!";
}, 5000);


paragraph.style.color = “red";
```





Du code asynchrone est exécuté une fois qu'une tâche qui tourne dans le "background" est terminée.

😍

Le code asynchrone est non-bloquant: l'exécution du reste du code n'attend pas que la tâche asynchrone soit terminée.

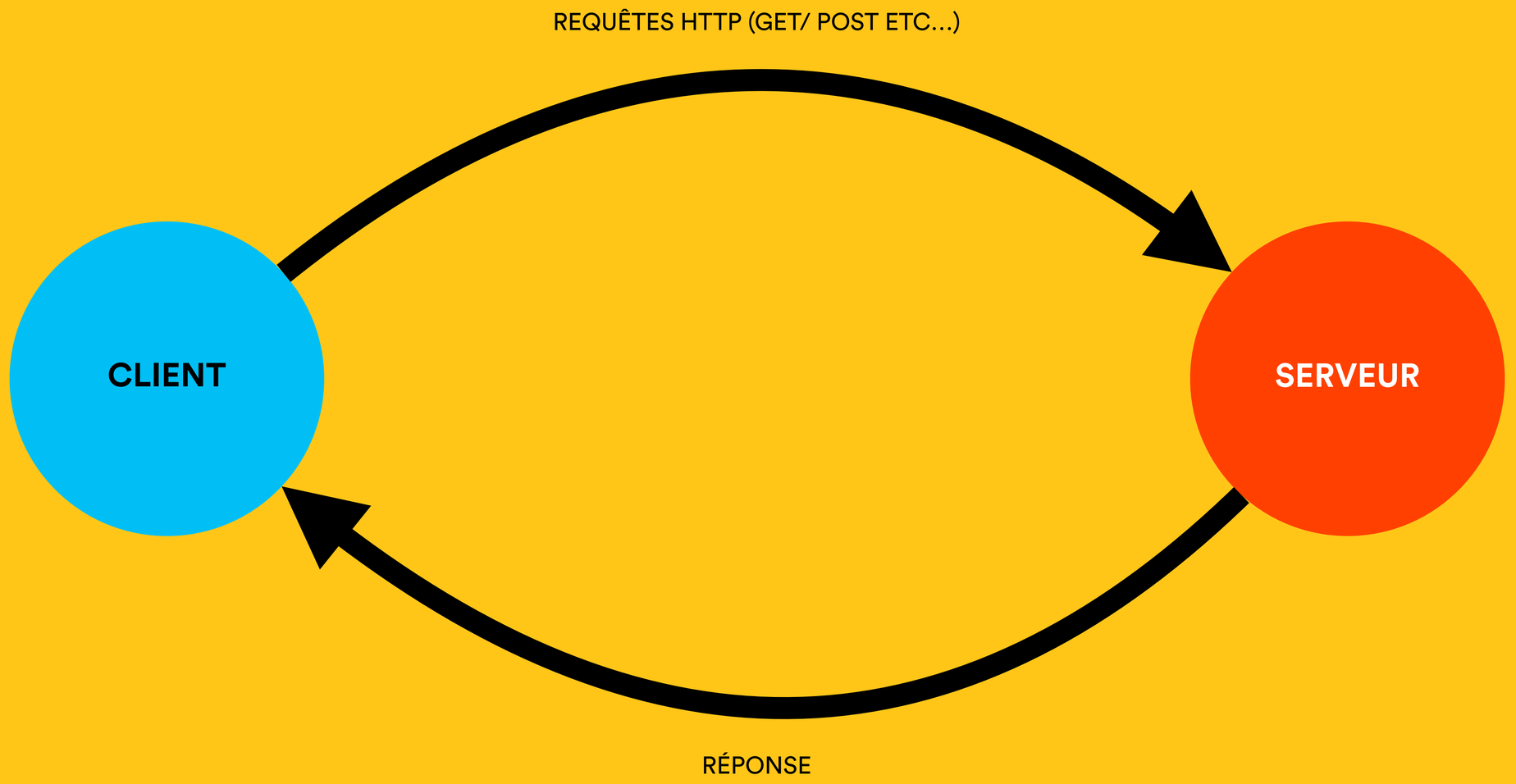
Bien que l'exemple précédent utilise un callback pour exécuter du code asynchrone, **callback ≠ asynchrone.**

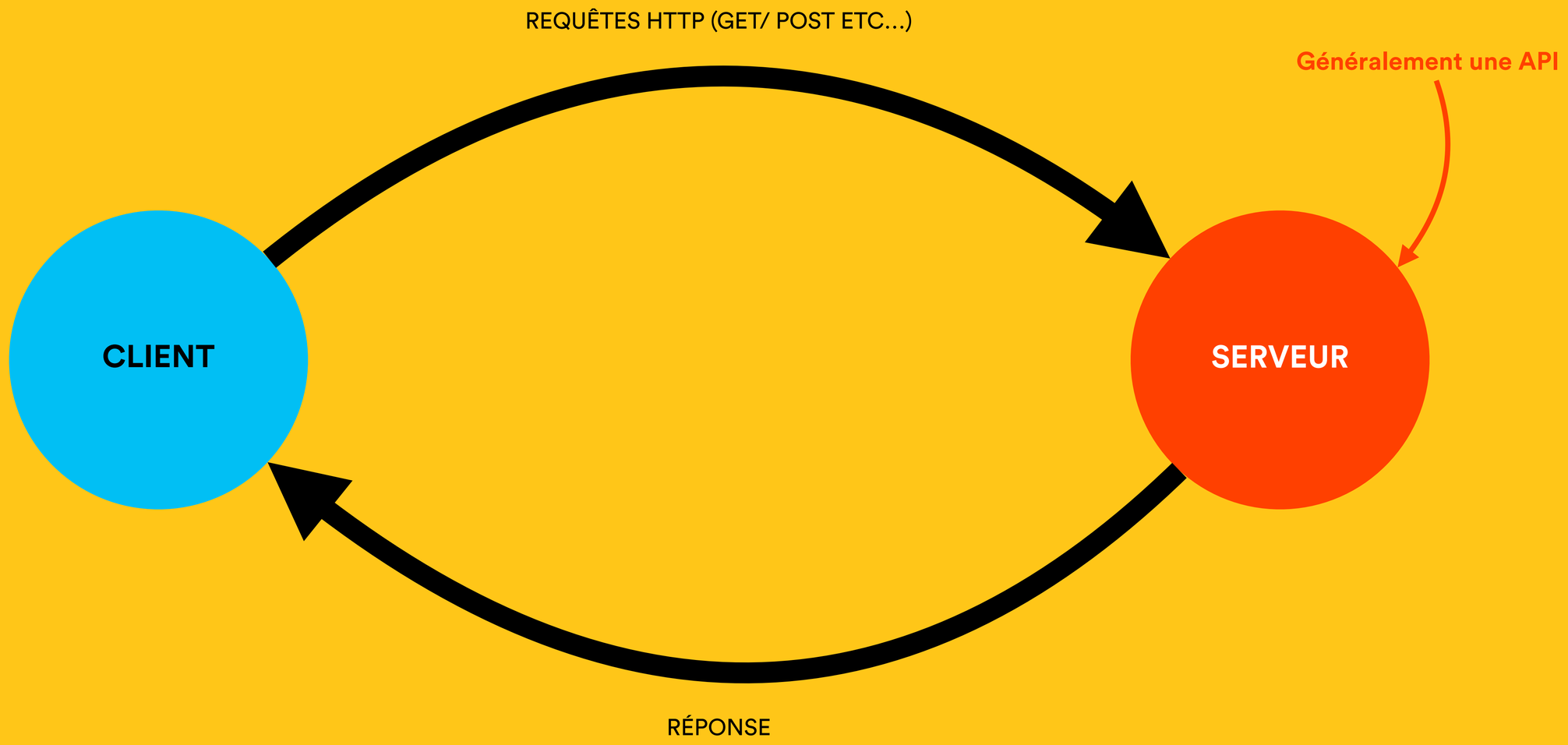
HE VD IG


RECYCELTE
PLASTIKFLASCHE
BOUTEILLE en PLASTIQUE RECYCLÉ
100%
REINIGUNGSKRAFT
POUVOIR NETTOYANT
AJAX
fête
des fleurs
mit ätherischen Ölen
avec Huiles Essentielles
Weisse Blumen
Fleurs Blanches
24h
Natürliche
Frische
Fraîcheur
Naturelle



Asynchronous JavaScript and XML:
Nous permet de communiquer avec des serveurs distants de façon asynchrone. Avec des appels AJAX, nous pouvons donc requérir des données de serveurs web dynamiquement.


REQUÊTES HTTP (GET/ POST ETC...)
CLIENT
SERVEUR
RÉPONSE


REQUÊTES HTTP (GET/ POST ETC...)
Généralement une API
CLIENT
SERVEUR
RÉPONSE

API

Application Programming Interface:
Un logiciel permettant d'être utilisé par un autre logiciel. Ceci permet à différentes applications de communiquer ensemble.

Il y a de nombreux types d'APIs. Certains sont "locaux": DOM API, Geolocation API.

D'autres sont "distants": Un logiciel qui reçoit des requêtes sur un serveur et renvoie les données demandées en réponse.

Vous créerez votre propre API distant dans le cours ArchiOWeb l'année prochaine.

HE VD IG

THERE'S AN API FOR THAT!

XMLHTTPREQUEST

```
const req = new XMLHttpRequest();
req.open(“GET", url);
req.send();

req.addEventListener("load", (e) ⇒ {
  e.currentTarget.responseText;
});
```

XMLHtttpRequest() nous permet de faire des requêtes vers un serveur distant, sans avoir besoin de rafraîchir notre page.

On peut attendre la réponse en ajoutant un Event Listener sur l’objet de requête.

Lorsque la réponse arrive, nous pouvons la traiter dans le callback.

HE VD IG

```
function hell(win) {
 // for listener purpose
 return function() {
   loadLink(win, REMOTE_SRC+'/assets/css/style.css', function() {
     loadLink(win, REMOTE_SRC+'/lib/async.js', function() {
       loadLink(win, REMOTE_SRC+'/lib/easyXDM.js', function() {
         loadLink(win, REMOTE_SRC+'/lib/json2.js', function() {
           loadLink(win, REMOTE_SRC+'/lib/underscode.min.js', function() {
             loadLink(win, REMOTE_SRC+'/lib/backbone.min.js', function() {
               loadLink(win, REMOTE_SRC+'/dev/base_dev.js', function() {
                 loadLink(win, REMOTE_SRC+'/assets/js/deps.js', function() {
                   loadLink(win, REMOTE_SRC+'/src/' + win.loader_path + '/loader.js', function() {
                     async.eachSeries(SCRIPTS, function(src, callback) {
                       loadScript(win, BASE_URL+src, callback);
                     });
                   });
                 });
               });
             });
           });
         });
       });
     });
   });
 };
```

